Hardware Design: Elevator Control Logic

ECE 150. Final Project
Prof. Risbud

Junbum Kim, Jonathan Weinrib
Dec 10th 2014

1. Abstract

The project will aim to reproduce a four-floor elevator control system for two elevators. To allow the users to move to the floors they would like to, there will be an up button on the first floor; an up button and a down button for floors 2-3; a down button on the fourth floor. In the elevator there will be four buttons for each floor, and the floor in which the elevator is will be represented by a seven-segment display.

User experience:

The user will press a button on a certain floor. An LED will light up to indicate that the button has been pressed and the request is in action, until the elevator comes. The elevator will give feedback to the user by showing which floor the elevator is on using the seven-segment display. The elevator will move a floor up at a frequency of 0.5Hz. When the elevator reaches the requested floor, it will hold for 5 seconds so that the door can open for the user to enter. During this interval the user will ride on the elevator, and press a button so that the elevator can move to the user's destination. An LED according to this button will light up until the transaction is completed.

Guideline for Tracking this Documentation:

There will be two parts to this project. Part. A will focus on implementing two working elevators. Part. B will aim to produce a selection algorithm that will instruct which Elevator to go where.

List of Equipment Used:

- 2 74LS193 4 bit L/R static shift registers
- 16 LEDs (2 for each 4 floors, 4 for each 2 elevators, and 2 to indicate elevator open time)
- 14 push buttons
- 8 SCL4543 SR Latches
- 2 CD4001 quad NOR gate
- 2 CD4027 JK flip-flops
- 2 CD4029 counters
- 6 CD4063 4bit magnitude comparator
- 6 CD4071 quad OR gate
- 10 CD4072 dual OR gate
- 14 CD4081 quad AND gate
- 6 CD4532 priority encoders
- 2 7 segment display
- 2 MAX6954 7 segment display drivers
- 4 555 timers (two different clock pulses for two elevators: one for the moving elevator and the other for the open/closed door)
- 20 breadboards
- 8 CD4063 4bit magnitude comparators

2. Design

Part A.

The first part of the design is the recreation of a working elevator. For part A, the primary inputs of the system are buttons: the six floor buttons, and four elevator buttons from each elevator. This was sent to the control logic, which will sort the floors and send them to one of the elevators. This was stored in the internal memory of the elevator, in which the elevator used as a guideline for which floor it should go to. The storage was implemented using a total of 26 SR latches (6 floors outside, 6 floors for each elevator's internal memory, and 4 elevator buttons on each elevator) so that we could control when to turn it on and off.

The golden principle was that the elevator had to always move in the direction the elevator was moving until it reaches the highest floor it had to go. Accordingly, the first thing that enabled the entire design is the direction of the elevators. JK flip-flops were used to store the elevator direction, and were toggled every time they had to flip the direction. In other words, the JK flip-flops were toggled every time the elevator had no more floors that it had to go to in that direction.

To control the elevator, which was made out of a 4bit bidirectional shift register 74LS194, we controlled the clock pulse that came from a counter CD4029 connected to a 1Hz astable 555 timer instead of controlling the pin settings (S0 and S1 of 74LS194). When there was no buttons pressed within the elevator internal button storage, we disabled the counter by inputting 1 using logic operations to the enabler of the counter. When the elevator first started moving, we had to guarantee that the elevator did not move for the first two seconds to prevent elevators from moving immediately when a button is pressed. This was implemented by using logic operations from K-Maps ($\overline{Q_A} \cdot (Q_B + Q_C + Q_D)$). When the elevator reached the designated floor, we had to guarantee that the elevator paused for 5 seconds, and then started moving after the 5 seconds. This was implemented using a 5 second monostable pulse triggered when the current elevator floor was equal to the button pressed.

To trigger the 555 monostable properly, the second goal was to ensure that the elevators stop at where it had to. There were three conditions in which the elevator should stop while going up:

1. Elevator buttons pressed (e.g. 0100)
2. Floor buttons pressed in the direction of elevator propagation (e.g. 001x for up buttons as there is no up button on the highest floor)
3. Greatest floor button pressed opposite to the direction of elevator propagation (Least floor while elevator is going down)

All three terms were recorded as 4 bit numbers. The first and the second conditions were simply ORed to each other and compared with the current floor the elevator was on, which was the outputs from the shift register. To accomplish the third condition, first the floor buttons were priority encoded the opposite directions from lowest to highest or highest to lowest depending on which direction the elevator was moving at. The resulting number was a 2-bit number because a priority encoder acted like a MUX. We had to decode this back into a 4-bit number. This was done by simple logic expression with two inputs and four outputs (W = 0, X = BA', Y = B'A, Z = B'A'), but in retrospect, it could have used a CD4028 decoder, which would have reduced a 4069 hex inverter chip. This resulting 4-bit number was then compared with the first two terms ORed using a CD4585 4bit magnitude comparator to

determine which of the two were greater. If the greatest down floor was greater, the result from the comparator was High and vice versa. This resulting high or low input was ANDed with the greatest opposite direction floor to ensure that the result of the comparator was only enabled when it should be. The resulting 4-bit number was ORed with the first two conditions ORed to one another. The resulting 4-bit number indicated where the elevators had to stop at. Each of the for bits were ANDed to the current floor of the elevator, and the four resulting bits were ORed to figure out when to trigger the monostable pulse.

For instance, let say the elevator was on the second floor going up (0100). If the resulting floors that elevator had to go to was (0011), the elevator will shift left (0010). As the four bits ANDed together will produce (0010), which is then ORed all together, which results in 1. This high sign triggered the monostable 555 timer, which disabled the astable 555 timer.

When the three conditions processed was 0000, indicating that the elevator had no place to go to, the JK flip-flop switched toggled so that the elevator changed its direction. The astable timer was disabled until the elevator got new instructions from the sorting algorithm.

Part B.
Selecting Algorithm:

The second part of this circuit was the decision making process. On each floor, there were either one or two external floor buttons that the user could press in order to call the elevator to his floor. On the first floor there was only an up floor button, and on the fourth floor there was only a down floor button; on the second and third floors there were both up and down floor button. Thus, the decision making process as to which elevator to take when a floor button is pressed evaluated six parameters (the six floor buttons). In order to minimize the number of chips and to prevent conflicting information, we chose to evaluate only one floor button at a time. A 555 timer in astable mode with a frequency acted as the clock pulse for a 4520 binary counter. This counter was connected to a 4028 BCD so that six individual states could be isolated. In retrospect this could have been done more efficiently with just a 4017 decade counter. The six states were then each individually ANDED with a 4081 chip so that at any point only two possibilities existed: If a floor button was pressed, at the point in the counter's cycle that corresponded to that button the output of the 4081 would be 1 while all other outputs would be 0. If that floor button was not pressed, all outputs would be 0 at that time.

These outputs were then compared with the outputs of the elevators using a 4585 4-bit magnitude comparator. The ability to do this was in fact another reason for choosing the method described in the above paragraph. Since an elevator can only be on one floor at a time, to compare the elevator in relation to buttons pressed, we found that it would be most useful to be able evaluate floor buttons pressed one at a time, even though in memory any buttons pressed are stored (meaning that if the down buttons on floor two and three were pressed the memory would hold 0110, but we wanted to be able to evaluate it in two stages 0100 and 0010). Four 4585 comparators were used; each elevator was compared to both up floor button outputs and down floor button outputs (meaning after the logic operations to get them in the desired form).

The basic theory behind the sorting algorithm was that an elevator that is not moving is usually a better fit to pick up a passenger than an elevator that is moving. This is true so long as an elevator that is moving up and is below an up floor button pressed is the best choice, as is an elevator that is moving down and is above a down floor button pressed. Otherwise, it is more logical to take the unoccupied elevator.

A 4028 decoder was used to distinguish and evaluate between the four possible cases of motion: Elevators A and B are both not moving, elevator A is moving and B is not moving, elevator A is not moving and B is moving, and both elevators are moving.

To evaluate the first case, namely, when both elevators are not moving, two 4028 BCD's were used to determine which elevator to take. One BCD evaluated the elevators in relation to up floors pressed, and one in relation to down floors pressed. In retrospect, these two BCD's gave identical outputs; thus, there was a way to combine them to reduce chips and simplify logic. However, the decision to evaluate the floors in this way was made later and the equivalence of the results was not realized until after the final circuit had been built. The most significant bit of each of these BCD's was the inverted value (using a 4069 hex inverter) of the first BCD (the one that evaluated whether or not the elevators were moving). Thus this bit acted as a selector; if this bit was zero (as in the first BCD said both elevators were moving), then these two decoders were selected. Otherwise, they were not selected, since any outputs with the most significant bit a 1 was ignored. The least significant bit was chosen to be the value of the first elevator ANDED with the output floor bits (again, one BCD for up floors and one BCD for down floors). For the BCD evaluating up floors, the most significant bit of the floors was grounded, since there was up floor button on the fourth floor. For the BCD evaluating down floors, the least significant bit was grounded, as there

was no down floor button on the first floor. Any output that corresponded to the least significant bit being 1 was determined to choose elevator A to accept the destination, since it meant that A was on the floor of the button pressed, as the AND operation would only output 1 if both the floor button being evaluated and the elevator floor were the same. The next least significant bit was chosen to be the second elevator ANDED with the floors in the same way. Any output of the decoder in which this bit was a 1 (and the first bit was not a 1) determined that elevator B would accept the destination. The third least significant bit was chosen to be A>Floor + A=Floor. This was implemented by evaluating the NOT of A<Floor. Any output that did not satisfy the first two conditions satisfied this condition, and the decision was made to send any such output to elevator B. While this did allow for certain possibilities to exist in which the wrong elevator would be chosen, in 13 out of the 16 possible cases the correct elevator would be chosen. This was a decision in our design to greatly reduce the number of chips in our circuit given our design.

The next two cases evaluated were when one elevator was moving and the other was not moving. Each of these cases was evaluated using a 4028 BCD. The most significant bit of the address of each of these two decoders was the inverted output of the first decoder – it thus acted as a selecting pin for that decoder. The next most significant bit was chosen to be that if the moving elevator was moving in the opposite direction to the direction of the floor button pressed, then the nonmoving elevator should be taken. This was evaluated by ANDING the direction of the moving elevator with the direction of the floor button. The last two address bits for the decoder was whether the moving elevator and the floor button pressed were in the same direction (done by ANDING the two variables). If this was true, another condition had to be evaluated. If, for example, the elevator and the floor were both in the up direction, which elevator should be chosen was determined by the location of the moving elevator. Using the 4585 comparators from before, if the moving elevator was below the floor button pressed, then that elevator was sent the destination (because it was moving up to an up floor button that was above it). Otherwise, the nonmoving elevator was chosen. In order to properly evaluate this final parameter, and to prevent false outputs being sent to the non-moving elevator, if the output of the 4585 comparator was zero, inverted that zero and ANDED it with the output of the BCD that had originally sent the process to the 4585. If this had not been done, any value in which the 4585 did not send the destination to the moving elevator would have sent it instead nonmoving elevator. However, we only wanted to send the nonmoving elevator in the case where the condition imposed explicitly failed. Since this only happened when the 4585 was actively evaluated (As in the BCD selected this output), we solved this possible error in this way. The same process was evaluated for the down direction.

The moving/not-moving case that had to be evaluated was when both elevators were moving. The ideal design for this case was to map out the different possible scenarios for which elevator was best in a any given case and send those values to a MUX. However, due to time constraints and chip constraints (the lab ran out of a lot of chips) a simplification in logic was chosen. Since so many of the variables between the cases in which only one elevator was moving and in the case where both were moving, we decided to divert the case of two moving elevators to one of the two cases in which only one elevator was moving. Since the latter case prioritizes the non-moving elevators, an ideal way to do this would have been to use a 4008 adder to see which elevator had the most destinations currently in its internal memory and to send the information to the BCD that prioritized the less busy elevator. This would have been implemented by inverting the bits in one of the elevators memory and adding that with the bits of the other elevators memory. Using two's complement addition, whether the sum was negative or positive would indicate which elevator had the most current destinations. However, given chip limitations (there were no

more adders), the decision was made to automatically reroute the destination to the comparator that prioritized elevator B.

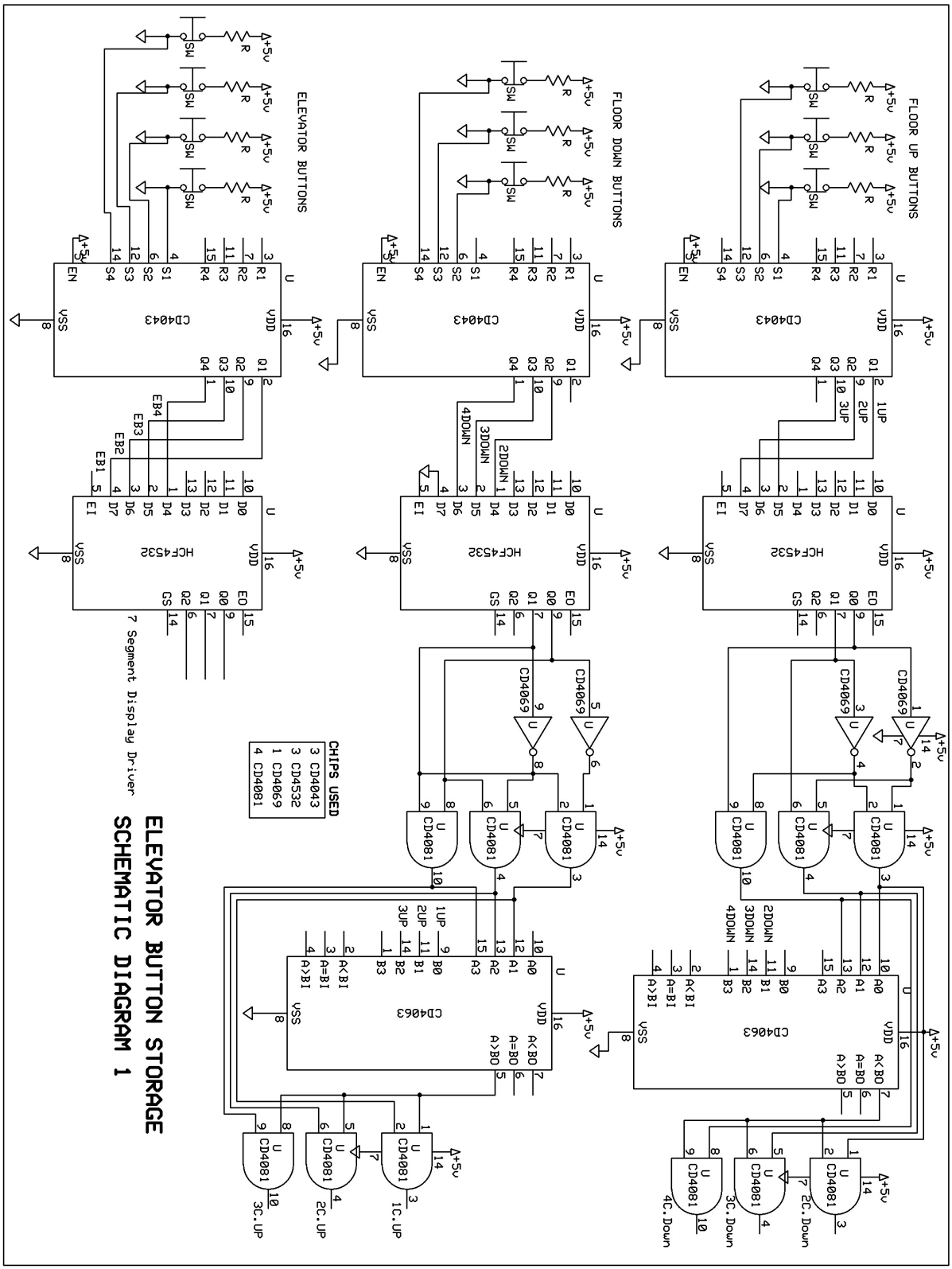
The final logic implementation in the decision-making process was integrating the decision with the elevator memory. All outputs that selected elevator A were ORed with 4072 chips (dual 4 input OR gates). The same was done for outputs that selected elevator B. The outputs of the 4072's were then ANDED with each of the floor button outputs (the outputs that evaluated only one floor at a time). Thus, if elevator A was selected to go to floor two to take the person who pressed the down button, only that destination would be sent to A. A value of one output from these AND gates acted as the set for the S/R latches for elevator memory. The also acted as the reset for the S/R latches that stored the floor buttons pressed (in order to stop evaluating them in the decision making process after it had already been sorted and delivered to a particular elevator).

3. Difficulties Encountered

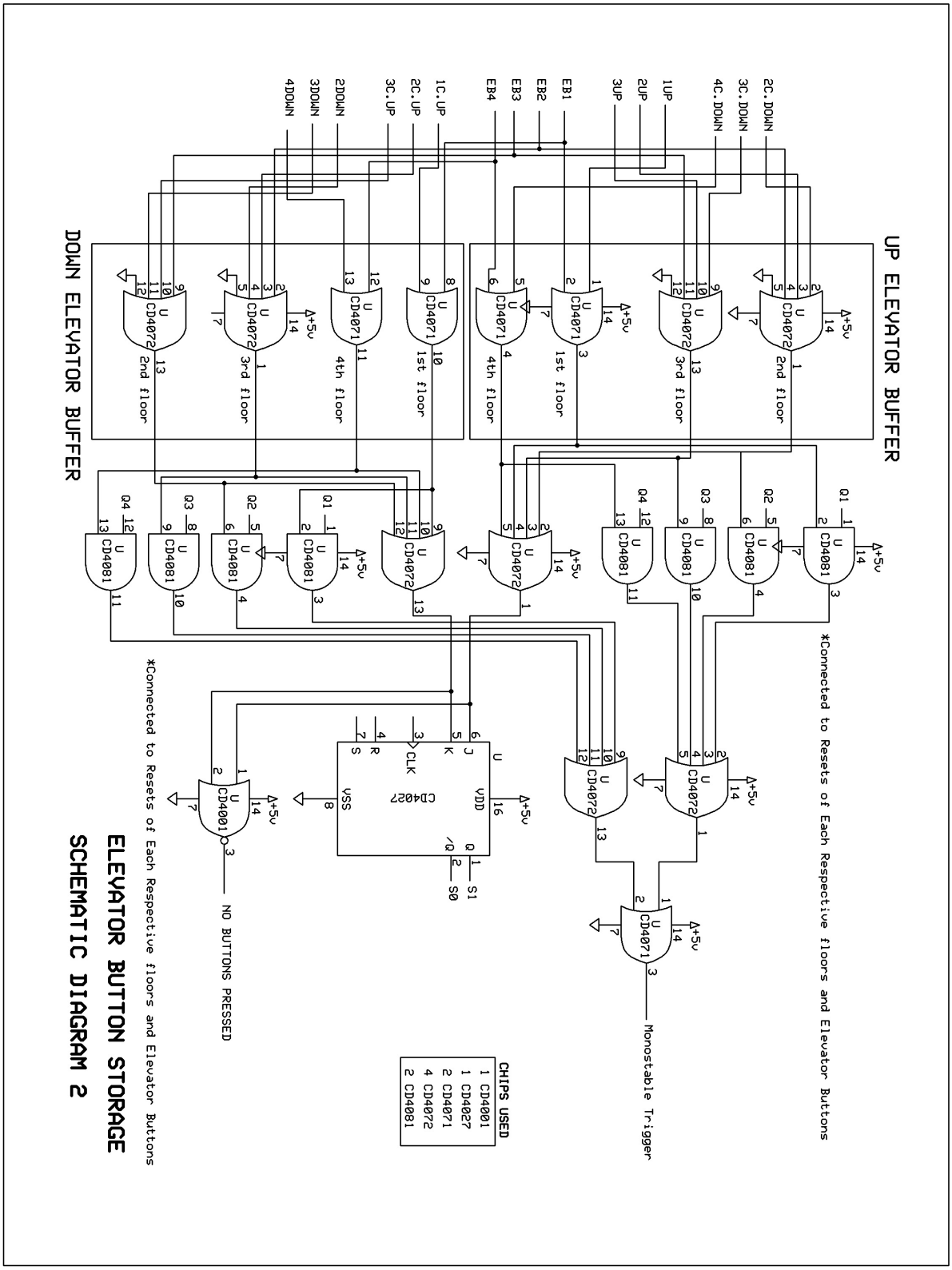
The hardest part of the design was coming up with what a systematic way to analyze the situation. While we were designing the elevator, it was hard to keep track of all the variables and how each related to one another. This caused us to waste some time, which was on one hand useful because we could test some functions of part B, but it did cost us a lot more time than what it should have.

Also, there was some loose connection between the breadboard and the wires that sometimes produced expected results but not in other times.

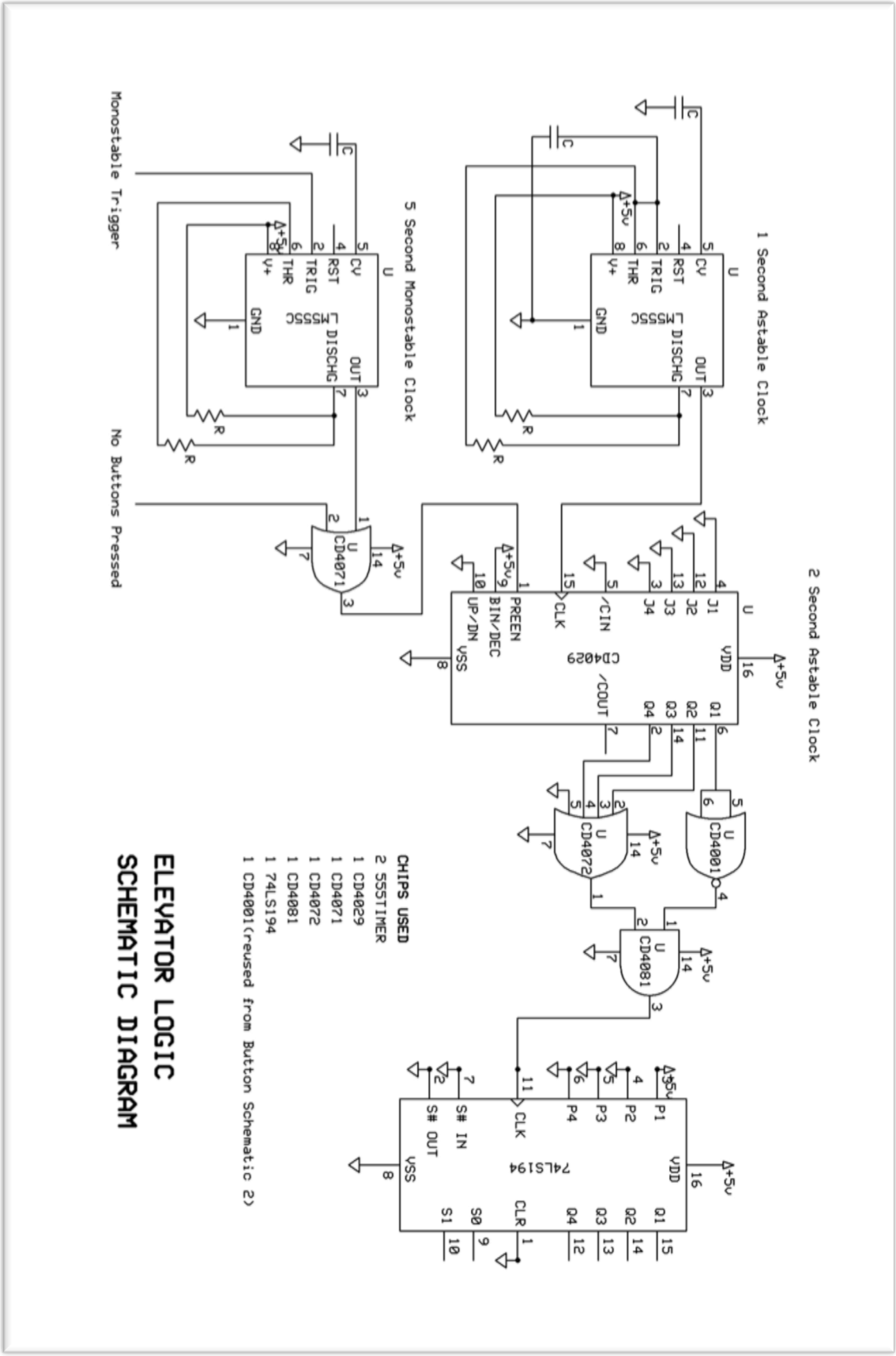
Schematic Diagram
Part A.
Button Schematic 1.



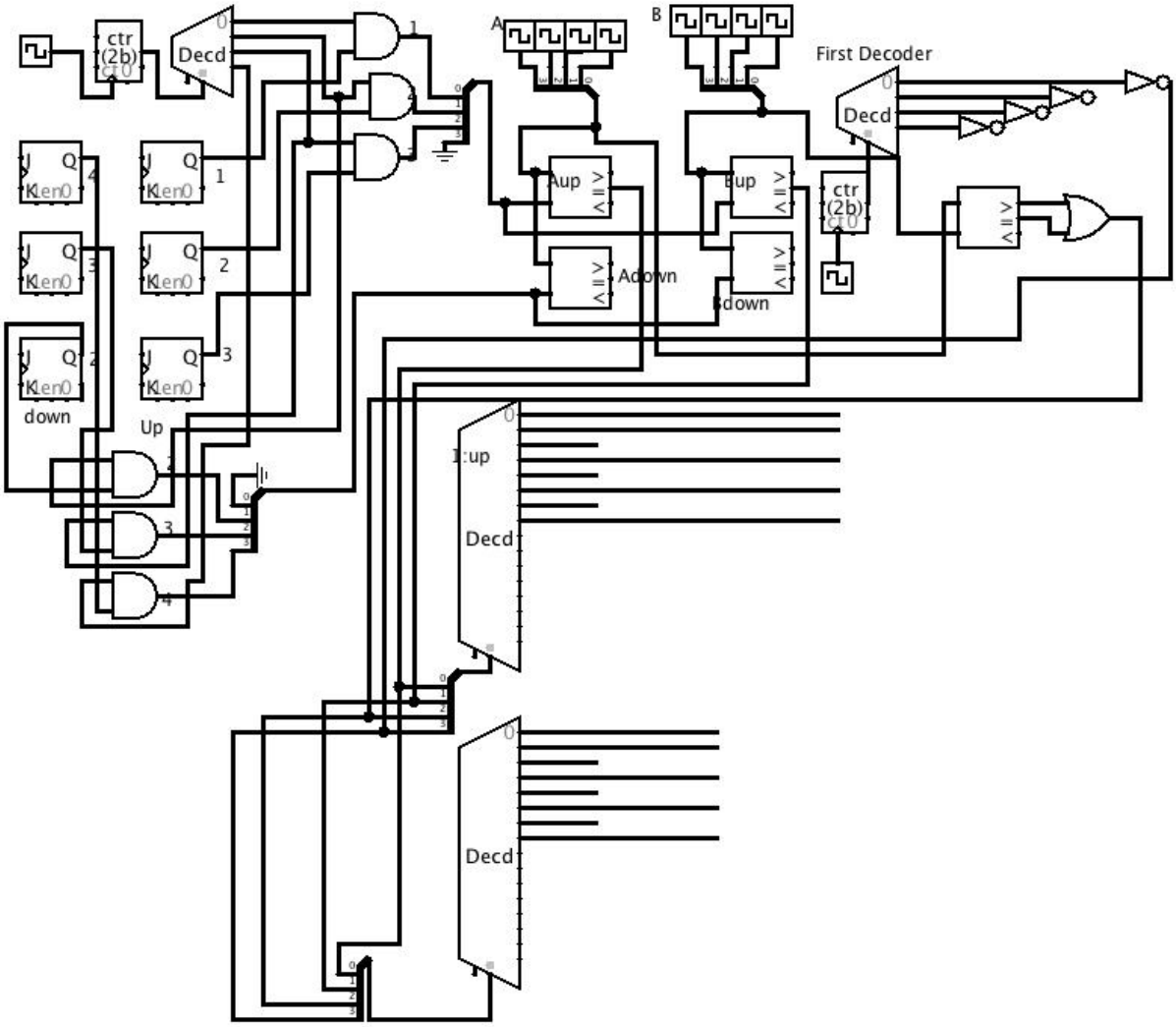
Button Schematic 2.

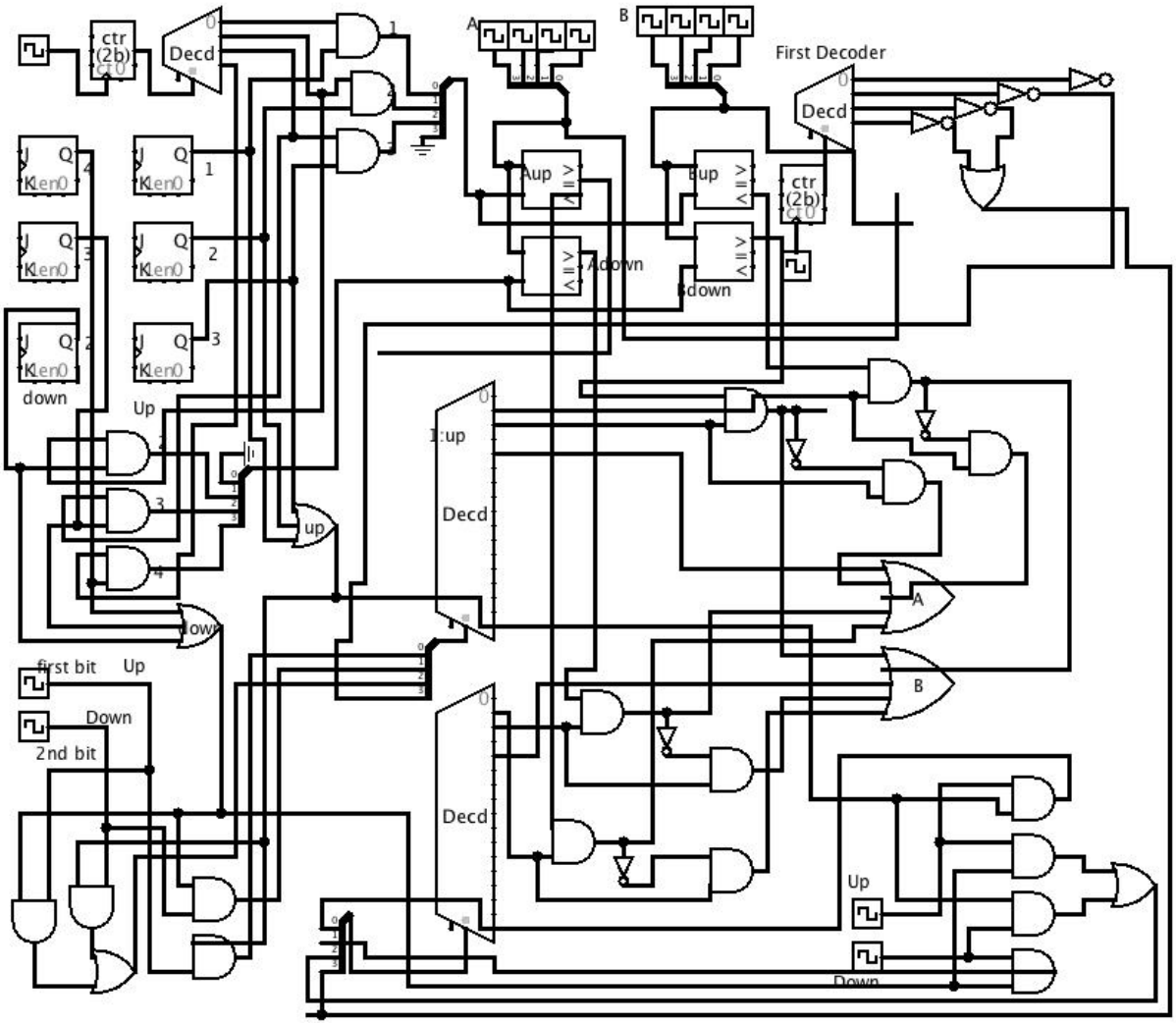


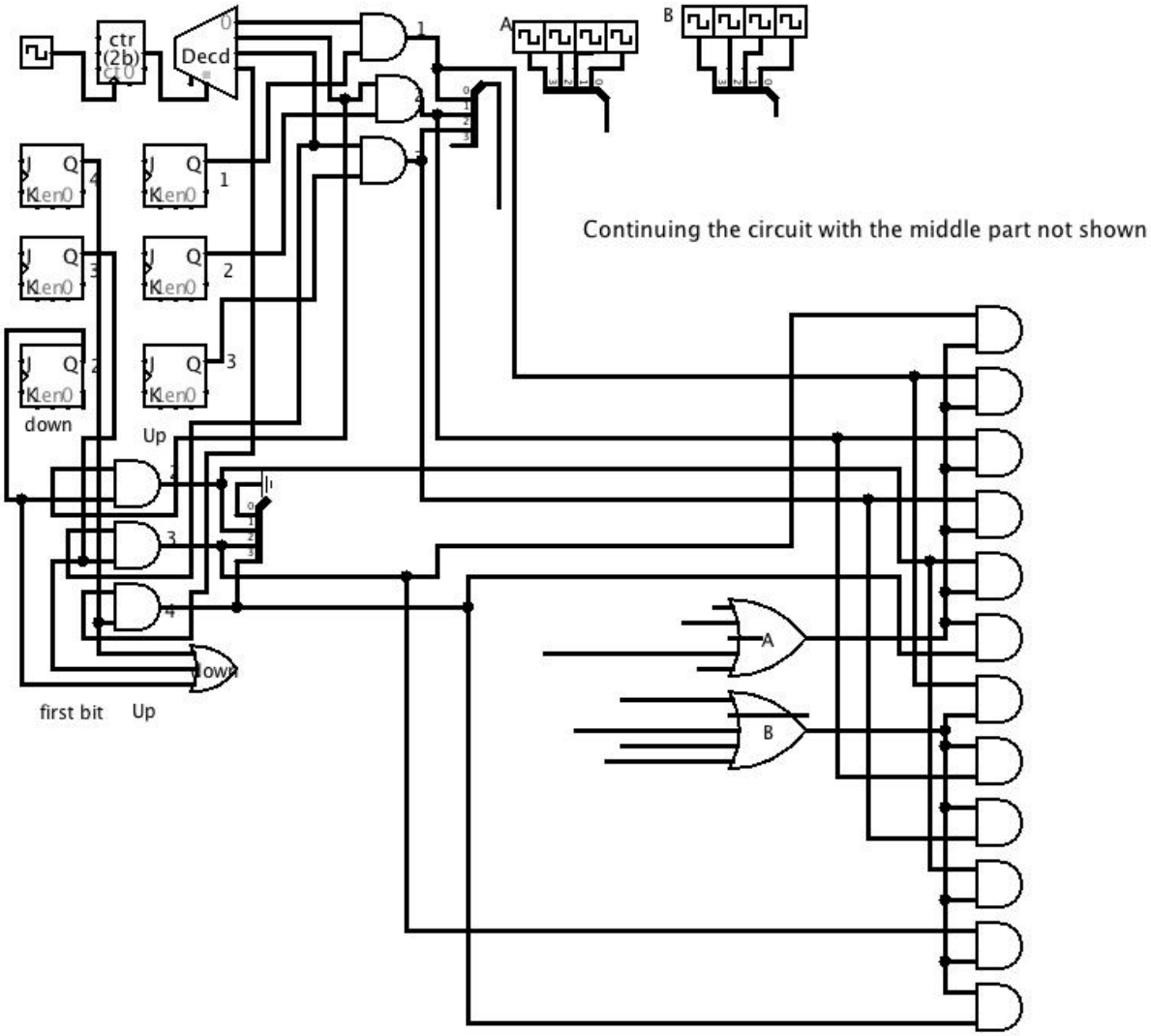
Elevator Schematic



Part B. Control Logic




ctr
(2b)
Decd
A
B
First Decoder
Decd
Aup
Adown
Bup
Bdown
ctr
(2b)
down
Up
up
Down
1:up
Decd
Decd
A
B
first bit
Up
Down
2nd bit
Up
Down


ctr
(2b)
Decd
A
B
Continuing the circuit with the middle part not shown
down
Up
first bit
Up
down
A
B